Language Models and Agents — Monsoon 2026

Individual Project [100 Marks]

Building and Analyzing Monolingual Transformer LMs
in Two Indian Languages

Starts: 12th August 2026

Final Deadline: 16th September 2026, 11:59 P.M. (5 weeks)

GitHub Classroom: `https://classroom.github.com/a/Q6gOCxoh`

Doubts: `https://hackmd.io/@CL3410/ryyiOBUIGe`

General Instructions

- Your project must be implemented in **Python** (PyTorch).
- Clearly label and organize your code, including comments that explain the purpose of each section and key steps. Structure repositories with headings, subheadings, and explanations as necessary.
- Test your code thoroughly before submission to avoid runtime errors or unexpected behavior. Training pipelines must support checkpoint resume (see Phase 2).
- Evaluation considers correctness, code quality, clarity of explanations, and how well you understand and apply course concepts.
- We are aware of the possibility of submitting late via platform hacks. Measures are in place accordingly; anyone caught attempting the same will receive a straight zero on the project.
- Post doubts on the course HackMD (`https://hackmd.io/@CL3410/ryyiOBUIGe`), not over private email unless necessary.

AI Tools Usage Instructions (Mandatory if Applicable)

We are aware how easy it is to write code and solve tasks with LLM services, but we strongly encourage you to figure out the answers yourself. If you use any AI tools such as ChatGPT, Gemini, Claude, etc., to assist in any part of this project:

- If you are unable to explain any part of the solution/code during evaluations, that solution/code will be considered plagiarized and you will be penalized. You must also be able to briefly explain how you modified or verified the AI-generated content.

Submission Instructions

- Submission is via **GitHub Classroom**. Accept the assignment at `https://classroom.github.com/a/Q6gOCxoh`.
- Submit each phase on a **separate branch**:

  - Phase 1 → branch `phase-1`
  - Phase 2 → branch `phase-2`
  - Phase 3 → branch `phase-3`

  Push your completed work to the corresponding branch **before** that phase's deadline. The state of the branch at the deadline is what will be graded for that phase.

- After a phase is graded, you **may** revise earlier-phase code in later branches (e.g. fix Phase 1 bugs while working on `phase-2`) based on feedback. Those revisions will **not** be re-evaluated: marks already awarded for a closed phase stay fixed.

- **Per-phase reports:** On each graded branch, include a short report (PDF or Markdown under `report/`) and/or notebooks that contain **all** results for that phase—tables, plots, loss curves, generated samples, attention heatmaps, and written analysis. Graders will not open external dashboards (e.g. Weights & Biases) or Drive folders to find figures; if it is not in the branch, it is not graded. Phase 3's report should consolidate the full project.

- **Large artifacts** (datasets, tokenized corpora, model checkpoints) must be uploaded to **Google Drive**. Put **shareable links** in the repository `README` (and keep permissions set so TAs can open them without requesting access). Do **not** commit large binary artifacts to git.

- Your repository must ultimately include at least:
  - Dataset collection and preprocessing code; train/val/test split definitions and dataset statistics.
  - Tokenizer training code and vocabulary / tokenizer files (or Drive links if too large, listed in the `README`).
  - Transformer implementation and model configuration files.
  - Google Drive links to pretrained checkpoints (Model H and Model L), plus training logs and loss curves in the repo.
  - Reasoning finetuning code, configs, Drive links to finetuned checkpoints, and reasoning metric results.
  - Evaluation scripts; perplexity / BPB tables; BLEU, chrF, ROUGE-L; generated samples; attention heatmaps and attention summary metrics.
  - Complete final report and a `README` with reproduction steps and all Drive links.

Submission Policy

The following policy applies on **GitHub Classroom**:

- **Strict phase deadlines:** Each phase is graded from the corresponding branch (`phase-1`, `phase-2`, `phase-3`) as of that phase's deadline. Late pushes to a phase branch receive **zero** for

that phase even if later branches are on time. There are no extensions by default.

- **Push frequently:** Do not dump all work in a single last-day commit. Commit and push regularly as you progress; history will be reviewed.
- **No re-evaluation of closed phases:** Feedback may be used to improve subsequent work, but a graded phase will not be re-scored after its deadline.
- **Commit Quality:** Commits must reflect actual progress. Non-informative or placeholder commits do not demonstrate progress.
- **Final submission:** Phase 3 / branch `phase-3` at the final deadline is your complete project snapshot; commit history on all branches may still be reviewed.

Guidelines for Implementation and Code Design

1. Prefer clear modular (preferably object-oriented) design: separate data, model, training, evaluation, and visualization code.
2. Do **not** use pretrained language models or pretrained tokenizers. Do **not** use HuggingFace Transformer model classes. Implement the architecture with basic PyTorch components. Do **not** train a single multilingual model with a shared vocabulary.
3. Use appropriate visualization libraries (e.g. matplotlib, seaborn, or plotly). Each plot must include a title, x-label, y-label, and legend (if applicable). Plots missing labels or legends may receive zero marks for that component.
4. Keep visualization and computation logic in separate functions.
5. Add docstrings to methods explaining what they do.
6. Log hyperparameters and training configs alongside checkpoints so runs are reproducible from the `README`.
7. **Repository layout.** Organize the repo by language, with each language directory self-contained. Example:

```
<repo>/
README.md                 # reproduction steps + Google Drive links
<language_1>/             # e.g. hindi/
tokenizer/
model/
train/
eval/
configs/
...
<language_2>/             # e.g. <your-low-resource-language>/
tokenizer/
model/
train/
```

```
    eval/
    configs/
    ...
report/                    # per-phase reports + all plots/tables/heatmaps
```

Put Google Drive links for datasets and checkpoints in the top-level `README`. Subfolder names inside each language directory may differ, but the two languages must remain clearly separated. Keep every figure and table that you want graded inside `report/` (or notebooks) on the corresponding phase branch.

Overview

In this individual project you will build, train, and analyze **two completely independent** decoder-only Transformer language models from scratch—covering corpus collection, tokenizer training, architecture implementation, pretraining, finetuning on reasoning tasks, evaluation, and attention analysis.

Choose one **higher-resource** non-English Indian language and one **lower-resource** language from the allowed list in Phase 1. For each language you will create a separate dataset, train a separate tokenizer and vocabulary, and pretrain a separate ∼25M-parameter monolingual model. The two resulting models must not share data, tokenizer, vocabulary, or weights. The project is designed for student-level compute (e.g. free Google Colab GPUs) and must support checkpoint-based training recovery.

Project at a Glance

Model H	Monolingual LM for a higher-resource **non-English Indian** language
Model L	Monolingual LM for an allowed lower-resource Indian language (see Phase 1)
Independence	Separate dataset, tokenizer, vocabulary, and weights for each model
Architecture	Decoder-only Transformer (∼25M params each)
Pretraining tokens	Target ∼500M tokens **per model** (independent corpora)
Finetuning	Synthetic comparative / basic reasoning data in each model's language
Vocabulary	Separate tokenizer/vocab per model

These are **not** multilingual models and **not** variants of one shared run. Do not mix languages, share a vocabulary, or reuse checkpoints across Model H and Model L.

Checkpointing is mandatory. Colab sessions may terminate unexpectedly. Your pipeline must save and resume from intermediate checkpoints that include model weights, optimizer state, scheduler state, training step, and configuration. A run that cannot resume after interruption will lose marks.

Contents, Marks, and Deadlines

Project starts on **12th August 2026**. Phase deadlines below are at **11:59 P.M.** on the stated date.

#	Phase	Duration	Deadline	Marks
1	Data Collection and Tokenizer Construction	1 week	19 Aug 2026	25
2	Model Implementation, Pretraining, and Evaluation	2.5 weeks	5 Sep 2026	40
3	Reasoning Finetuning, Attention Analysis, and Final Report	1.5 weeks	16 Sep 2026	35
	Total / Final submission	**5 weeks**	**16 Sep 2026**	**100**

Complete phases in order. Phase 3's deadline is also the final project deadline.

1. Phase 1: Data Collection and Tokenizer Construction [25 Marks]

Deadline: 19th August 2026, 11:59 P.M.

Build **two independent** dataset and tokenizer pipelines—one for the higher-resource language and one for the lower-resource language.

1.1 Language Selection

- **Higher-resource (Model H):** any **non-English Indian** language with relatively abundant public text. Justify briefly.
- **Lower-resource (Model L):** must be exactly one of: Assamese, Bhojpuri, Bodo, Dogri, Konkani, Maithili, Manipuri, Mizo, Nepali, Sindhi.

1.2 Dataset Requirements

Each model has its **own** monolingual corpus. Target approximately **500M training tokens per language** after tokenization. If the lower-resource language cannot reach 500M from public sources, collect the maximum feasible amount, report the exact token count, and justify the shortfall.

You may use public corpora (e.g. Hugging Face), but for **both** languages at least **20%** of the final training tokens must come from **manual collection** (e.g. OCR from books/PDFs, scraping + cleaning pages you gather yourself, typed/transcribed text). Report the manual vs. downloaded token split in your dataset statistics.

Model	Language tier	Corpus
Model H	Non-English Indian (higher-resource)	Own dataset; ∼500M tokens; ≥20% manual
Model L	Allowed lower-resource list above	Own dataset; ∼500M tokens; ≥20% manual

Do **not** share documents across corpora or concatenate languages. Maintain separate train/validation/test splits and statistics for each language. Required work (for each language): collect

sources; remove invalid/duplicate data; normalize Unicode; create splits; report corpus statistics (size, sources, cleaning steps, manual fraction).

1.3 Tokenizer Training

Train a **separate** tokenizer from scratch for each language (BPE, SentencePiece, or Unigram). Pretrained tokenizers are not permitted. Recommended vocabulary size is in the tens of thousands **per model**; choose using fertility / unknown-token rate on held-out text. Vocabularies must not be shared.

Report, for each language: vocabulary size, token-frequency statistics, average characters per token, tokenization examples, and unknown-token statistics.

Deliverables

1. Dataset collection scripts (both languages).
2. Dataset preprocessing pipelines.
3. Per-language dataset statistics reports.
4. Per-language train/validation/test splits.
5. Tokenizer training code (both languages).
6. Vocabulary files (one per language).
7. Tokenizer model files (one per language).

2. Phase 2: Model Implementation, Pretraining, and Evaluation [40 Marks]

Deadline: 5th September 2026, 11:59 P.M.

Implement a decoder-only Transformer from scratch, pretrain Model H and Model L independently, and evaluate both as language models.

2.1 Required Architecture

Build a decoder-only (GPT-style) Transformer language model yourself in PyTorch. You may use primitive layers such as `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout`, but you may **not** use `nn.Transformer*` modules, HuggingFace model classes, or any pre-built attention block. The point of this phase is that you understand every tensor operation in the forward pass and can explain it during evaluation.

The forward pass must consist of the following stages.

1. Input representation

- **Token embeddings:** map token ids to dense vectors using an embedding matrix of shape (vocabulary size $\times$ model dimension).
- **Positional embeddings:** self-attention is permutation invariant, so position must be injected explicitly. Use learned absolute positional embeddings or sinusoidal encodings (or a relative scheme such as RoPE if you prefer); state which you chose and why, and explain how it constrains the maximum sequence length your model can handle.

- Combine the two (typically by addition) and apply embedding dropout.

2. Multi-head causal self-attention

Implement attention from first principles. For an input sequence $X \in \mathbb{R}^{T \times d}$:

- Project X into queries, keys, and values, $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- Reshape into h heads so each head works in a $d_k = d/h$ dimensional subspace; be explicit about the reshape and transpose operations that turn (B, T, d) into (B, h, T, d_k) and back.
- Compute scaled dot-product attention per head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

  and explain why the $1/\sqrt{d_k}$ scaling is needed.
- **Causal masking:** M is an additive mask with 0 on allowed positions and $-\infty$ (a large negative number) on future positions, so that position t attends only to positions $\leq t$. Implement the mask yourself (e.g. an upper-triangular mask applied before softmax) and verify empirically that the model cannot see the future—for example, show that changing token $t{+}1$ does not change the logits at position t.
- Concatenate the heads and apply the output projection W_O.

3. Transformer block

Each block must contain:

- a multi-head causal self-attention sublayer,
- a position-wise feed-forward network (two linear layers with a non-linearity such as GELU or SwiGLU, with an inner dimension larger than the model dimension),
- **residual connections** around both sublayers, and
- **layer normalization** (pre-norm or post-norm—state which you used and why pre-norm is generally more stable for deep stacks).
- dropout in the appropriate places (attention probabilities, sublayer outputs).

Stack N such blocks, followed by a final normalization layer.

4. Output head and objective

- A linear projection from the model dimension to vocabulary size produces next-token logits. You may tie this with the input embedding matrix; if you do, say so and explain the parameter saving.
- Train with the causal language modeling objective: cross-entropy between the logits at position t and the token at position $t{+}1$, averaged over all positions.

Configuration and reporting

Choose your own hyperparameters (model dimension, number of layers, number of heads, feed-forward width, context length, dropout) subject to a target of approximately ∼25M trainable parameters per model. Record them in a config file, report the exact parameter count for each model, and briefly justify the depth/width trade-off you picked given the compute budget. Vocabularies remain separate and come from Phase 1, so the embedding and output layers may differ in size between Model H and Model L.

You must be able to explain, for any component above, what it computes, what its tensor shapes are, and what would break if it were removed.

2.2 Pretraining

Pretrain Model H and Model L independently on their respective monolingual corpora (target ∼500M tokens each) using AdamW, learning-rate scheduling, validation evaluation during training, and checkpoint saving. Each checkpoint must contain model weights, optimizer state, scheduler state, current training step, and a configuration file. Do not initialize one model from the other.

2.3 Language Modeling and Attention Evaluation

Evaluate Model H and Model L separately on their own held-out sets. Report every metric for **both** models side by side.

Intrinsic language-modeling metrics

- Validation cross-entropy loss and perplexity ($\text{PPL} = e^{\text{CrossEntropyLoss}}$).
- Bits-per-byte (BPB) on the same held-out text, to make the two languages more comparable when tokenizers differ.
- Discuss the gap between Model H and Model L (data size/quality, script, tokenizer fertility, etc.).

Generation quality

For each model, generate continuations from held-out prefixes using greedy decoding and temperatures 0.5, 1.0, and 1.5. Against reference continuations from the held-out set (or teacher-forced targets), report:

- **BLEU** (corpus-level; state n-gram order, typically BLEU-4),
- **chrF** or chrF++,
- **ROUGE-L**,

and briefly explain why each metric is or is not informative for open-ended LM generation in your languages.

Also report simple fluency / diversity diagnostics:

- repetition rate (e.g. fraction of repeated n-grams),
- Distinct-1 / Distinct-2 (unique unigrams / bigrams over generated tokens),
- qualitative notes on coherence and language correctness.

Attention analysis

Because you implemented multi-head attention yourself, evaluate what it learned:

- **Attention heatmaps** for each model: at least one early layer, one late layer, and multiple heads. Show query vs. key positions with a color scale for attention weights; include example sentences in the model's language.
- **Attention entropy** per head/layer (higher entropy $\approx$ more diffuse attention).
- **Mean attention distance** (how far, on average, each query attends along the sequence) to contrast local vs. long-range heads.
- Short discussion: which heads look local/positional, which look content-based, and how Model H and Model L differ.

Deliverables

1. Transformer implementation (MHA, positional embeddings, causal mask).
2. Model configuration files (H and L).
3. Parameter counts; training scripts; Drive links to checkpoints.
4. Training logs and loss curves.
5. PPL / BPB tables; BLEU, chrF, ROUGE-L results.
6. Generated samples and diversity / repetition stats.
7. Attention heatmaps plus entropy / distance summaries.
8. Resource-level comparison write-up.

3. Phase 3: Reasoning Finetuning, Attention Analysis, and Final Report [35 Marks]

Deadline: 16th September 2026, 11:59 P.M. (final project deadline)

Finetune each model on reasoning tasks, analyze attention, and submit the final scientific comparison.

3.1 Reasoning Finetuning

Finetune **each** pretrained model independently on reasoning tasks in **its own language**. Do not share finetuning data or mix languages.

Synthetic reasoning dataset (required)

Build your **own synthetic** finetuning dataset in the language of each model (one corpus for Model H, one for Model L). Do not download an existing reasoning benchmark as the primary finetune set—generate examples programmatically (or with carefully controlled templates) so you know the ground-truth labels.

Focus on **comparative and basic reasoning**, especially transitive / ordering comparisons. Examples of the intended style (render the same ideas in each model's language, not in English only):

- Given $A > B$ and $B > C$, which is smallest / largest?
- Given heights, ages, prices, or quantities for two or three entities, answer comparison questions (greater, smaller, equal).
- Short multi-hop comparisons that require chaining inequalities (e.g. $A > B$, $B > C \Rightarrow$ relation between A and C).

Requirements:

- Text must be in the target language's script and natural phrasing for that language (not English prompts with foreign names only).
- Include train / validation / test splits; report size, template variety, and how you avoid train–test leakage (e.g. held-out entity names or held-out relation patterns).
- Release the generation scripts so the dataset is reproducible.

Protocol

- Start from that language's own pretrained checkpoint.
- Keep that language's tokenizer and vocabulary fixed.
- Document all hyperparameters.
- Save finetuned checkpoints in the same resume-capable format as pretraining; put Google Drive links in the `README`.

Report pretrained vs. finetuned accuracy (or exact match) on the synthetic test set, qualitative successes/failures, and a comparison between Model H and Model L on these reasoning tasks.

3.2 Attention Analysis (post-finetune)

Reuse the Phase 2 attention toolkit on the **finetuned** checkpoints. Compare pretrained vs. finetuned heatmaps for at least one early and one late layer per model. Comment on whether finetuning changed local vs. long-range attention or head specialization, especially on comparative-reasoning prompts.

3.3 Final Report

Your Phase 3 report should consolidate the project (including key Phase 1–2 figures/tables as needed) and answer:

1. How did data scale and quality differ between Model H and Model L?
2. How do language-modeling and reasoning results compare across the two resource tiers?
3. What tokenizer / corpus factors most affected the lower-resource model?
4. What evidence explains the observed differences?

Include a `README` with reproduction steps.

Deliverables

1. Reasoning data prep / generation scripts, finetune scripts, configs.
2. Finetuned checkpoints (H and L) via Google Drive links.
3. Finetuning logs, metrics, qualitative examples.
4. Pretrain vs. finetune attention comparison.
5. Final comparison tables and error analysis.
6. Complete final report and reproducibility instructions.

Final Submission and Constraints

Submission checklist. Your final submission must contain:

- Dataset and preprocessing code (both).
- Separate tokenizers and vocabularies.
- Transformer implementation.
- Google Drive links to pretrained checkpoints (H and L).
- Google Drive links to finetuned checkpoints (H and L).
- Training and finetuning logs.
- Evaluation scripts and results.
- Attention visualizations.
- Final report.
- README with reproduction steps and all Drive links.

Hard constraints. No pretrained models; no pretrained tokenizers; no HuggingFace Transformer models. Build **two fully independent monolingual** LMs (separate data, tokenizer, vocabulary, and weights). Do not train a multilingual model or share artifacts across languages. The main graded project is **not** an architecture bake-off (e.g. LayerNorm vs. RMSNorm); the optional bonus ablation below is separate. Positional embeddings, multi-head attention, and causal masking must be student-implemented. All experiments must be reproducible.

Bonus (optional)

Ablation—no positional embeddings. As an optional bonus in the **same** project repository, pick **one** of your two languages and retrain a model with positional embeddings removed. Run the **full Phase 2 evaluation suite** on that ablated model (perplexity / BPB, generation metrics including BLEU / chrF / ROUGE-L, diversity diagnostics, and attention heatmaps / entropy / distance) and compare it to your standard model for that language. Bonus marks from this ablation can be used to recover marks lost elsewhere in the project; they do not raise the score above the project maximum of 100. Document the ablation setup, results, and a short explanation of what breaks without position information.